Assignment 7 Report

Padding Oracle Attack on AES-CBC Encryption

Course/Lab: CryptoLab – Lab 7
Assignment: Padding Oracle Attack
Programming Language: Python

---

1. Aim

To implement and demonstrate a Padding Oracle Attack against AES-CBC encryption to recover the original plaintext without knowing or accessing the AES encryption key.

The experiment demonstrates how an attacker can exploit a padding-validation oracle to recover plaintext bytes by modifying ciphertext blocks and observing whether the resulting decrypted plaintext contains valid PKCS#7 padding.

2. Objectives

- Understand the working of AES-CBC encryption and decryption.
- Understand PKCS#7 padding and its importance in block cipher encryption.
- Understand the concept and working of a Padding Oracle Attack.
- Implement a padding oracle that indicates whether decrypted ciphertext has valid padding.
- Modify ciphertext blocks to generate suitable padding conditions.
- Query the padding oracle to recover plaintext bytes from right to left.
- Recover the complete plaintext without using or accessing the AES encryption key during the attack.
- Count the total number of oracle queries required to recover the plaintext.
- Verify the recovered plaintext against the original plaintext.
- Analyze the security implications of padding oracle vulnerabilities.
- Suggest appropriate security measures to prevent padding oracle attacks.

3. Problem Statement

The objective of this assignment is to demonstrate how plaintext can be recovered from an AES-CBC encrypted message without knowing the encryption key.

AES-CBC encrypts plaintext in fixed-size blocks and uses an Initialization Vector (IV) to process the first block. When the plaintext length is not an exact multiple of the AES block size, padding is added before encryption.

In this assignment, PKCS#7 padding is used. A padding oracle indicates whether the decrypted ciphertext contains valid padding.

An attacker can modify the ciphertext and submit the modified ciphertext to the oracle. By observing the oracle's responses, the attacker can infer information about the decrypted plaintext.

The attack recovers plaintext bytes from right to left by modifying the preceding ciphertext block and searching for values that produce valid padding.

The AES key must not be accessed or used by the attack procedure. The final recovered plaintext is verified against the original plaintext to demonstrate the success of the attack.

4. Theory and Important Concepts

4.1 Advanced Encryption Standard (AES)

Advanced Encryption Standard (AES) is a symmetric-key encryption algorithm used to protect information.

In symmetric encryption, the same secret key is used for encryption and decryption.

AES operates on fixed-size blocks of **128 bits, or 16 bytes**. AES supports key lengths of 128, 192, and 256 bits.

In this assignment, AES is used in Cipher Block Chaining (CBC) mode.

4.2 Cipher Block Chaining (CBC) Mode

CBC is a block cipher mode in which each plaintext block is XORed with the previous ciphertext block before encryption.

For the first block, the Initialization Vector (IV) is used instead of a previous ciphertext block.

The CBC encryption formula is:

$$Ci = EK(Pi \oplus Ci-1)$$

For the first block:

$$C1 = EK(P1 \oplus IV)$$

Where:

- **Pi** represents the plaintext block.
- **Ci** represents the ciphertext block.
- **EK** represents AES encryption using key KK.
- **IV** represents the Initialization Vector.
- $\oplus$ represents the XOR operation.

During decryption, the plaintext is recovered using:

$$Pi = DK(Ci) \oplus Ci-1$$

For the first block:

$$P1 = DK(C1) \oplus IV$$

This relationship is important for the Padding Oracle Attack because modifying the previous ciphertext block changes the corresponding plaintext block after decryption.

The attacker does not need to know the AES key to exploit this relationship.

4.3 PKCS#7 Padding

AES operates on blocks of 16 bytes. Therefore, plaintext must be padded when its length is not a multiple of 16 bytes.

PKCS#7 is a padding scheme that adds bytes to the plaintext until its length becomes a multiple of the block size.

Each padding byte contains the number of padding bytes added.

For example, if the plaintext requires three additional bytes, the padding is:

```
03 03 03
```

If the plaintext requires five additional bytes, the padding is:

```
05 05 05 05 05
```

If the plaintext length is already a multiple of 16 bytes, a complete block of padding is added.

During decryption, the padding is checked and removed to recover the original plaintext.

Incorrect padding can indicate that the decrypted data has been modified or corrupted.

4.4 Padding Oracle

A padding oracle is a function that reveals whether the padding of decrypted ciphertext is valid.

In this assignment, the oracle returns a Boolean result:

- **True:** The decrypted ciphertext contains valid PKCS#7 padding.
- **False:** The decrypted ciphertext contains invalid PKCS#7 padding.

The attacker does not receive the AES key or directly obtain the plaintext from the oracle.

Instead, the attacker uses the oracle's responses to determine which ciphertext modifications produce valid padding.

Repeated queries allow the attacker to infer plaintext bytes.

4.5 Padding Oracle Attack

A Padding Oracle Attack exploits differences in the responses of a system when it processes ciphertext with valid and invalid padding.

The attack works because CBC decryption combines the decrypted ciphertext block with the previous ciphertext block using XOR.

By modifying the previous ciphertext block, an attacker can influence the resulting plaintext block.

The attacker systematically changes ciphertext bytes and queries the padding oracle until a valid padding response is obtained.

The recovered bytes are used to determine the intermediate decrypted block and then the original plaintext.

The attack proceeds from the rightmost byte to the leftmost byte because PKCS#7 padding is validated starting from the end of the plaintext block.

By repeating the process for all relevant blocks, the attacker can recover the complete plaintext without knowing the encryption key.

---

5. Methodology / Algorithm

The Padding Oracle Attack was implemented using Python.

The following steps describe the overall procedure.

1. Initialize the original plaintext.
2. Generate an AES key and Initialization Vector for the encryption demonstration.
3. Apply PKCS#7 padding to the plaintext.
4. Encrypt the padded plaintext using AES-CBC.
5. Implement a padding oracle that checks whether decrypted ciphertext has valid PKCS#7 padding.
6. Provide the ciphertext and IV to the attack procedure.
7. Select the ciphertext block whose plaintext is to be recovered.
8. Modify the preceding ciphertext block to influence the decrypted plaintext.
9. Begin the recovery process with the rightmost byte of the target plaintext block.
10. Modify the relevant ciphertext byte and query the padding oracle.
11. Repeat the process until the oracle returns a valid-padding response.
12. Use the successful response to infer the intermediate decrypted byte and recover the corresponding plaintext byte.
13. Repeat the process for the remaining bytes, moving from right to left.
14. Recover the plaintext bytes of the target block.
15. Repeat the procedure for the remaining ciphertext blocks.
16. Remove the recovered padding from the final plaintext.
17. Compare the recovered plaintext with the original plaintext.
18. Count the total number of oracle queries made during the attack.
19. Report whether plaintext recovery was successful.

Note: The actual byte-recovery procedure must use the precise XOR relationships between the modified ciphertext block, the intermediate decrypted block, and the target padding value.

---

6. Implementation

The assignment was implemented in Python using an AES cryptographic library.

The implementation consists of the encryption procedure, padding oracle, attack procedure, and verification logic.

6.1 AES-CBC Encryption

The encryption procedure takes the original plaintext, applies PKCS#7 padding, and encrypts the resulting data using AES-CBC.

The AES key and IV are generated for the encryption demonstration.

The ciphertext is produced before the attack begins.

During the attack, the AES key is not accessed by the plaintext-recovery procedure. The attacker uses only the ciphertext, IV, and padding oracle.

6.2 Padding Oracle Implementation

The padding oracle decrypts the submitted ciphertext and checks the validity of the PKCS#7 padding.

The oracle returns a Boolean value indicating whether the padding is valid.

The attack procedure uses these responses to test different modifications of the ciphertext.

The oracle acts as the source of information that allows plaintext recovery.

6.3 Ciphertext Modification

The attack modifies bytes of the ciphertext block preceding the target plaintext block.

Because CBC decryption uses XOR between the decrypted block and the preceding ciphertext block, changing the preceding ciphertext block changes the resulting plaintext.

The attack searches for modifications that cause the final plaintext bytes to match a valid PKCS#7 padding pattern.

The successful modifications are then used to infer the original plaintext bytes.

6.4 Plaintext Recovery

The plaintext-recovery procedure begins with the rightmost byte of a target plaintext block.

It systematically modifies the corresponding byte in the preceding ciphertext block and queries the padding oracle.

When a suitable response is obtained, the attack uses the padding information to recover the plaintext byte.

The procedure continues from right to left until the entire block has been recovered.

The same procedure is repeated for the remaining blocks.

6.5 Verification

After recovering the plaintext, the program compares it with the original plaintext used for encryption.

If both plaintexts match, the recovery is considered successful.

The program also records the total number of oracle queries required during the attack.

---

7. Execution and Output

The program was executed using the following command in the terminal:

```
None
python main.py
```

The program successfully encrypted the original plaintext, performed the Padding Oracle Attack, recovered the plaintext, and verified the result.

7.1 Program Output

The execution output was:

```
None
Original plaintext: Padding oracle attack demonstration.
IV: 3830e9bd6c86271e2297d315df4683c5
Ciphertext:
2b1ddfc290fc1e868471b3f78f634475ac1a5bc6240946847313369721 4add07ed1
b18547b06e8b52dad2245dbbe0663
Recovered plaintext: Padding oracle attack demonstration.
Oracle queries: 5952
Recovery successful: True
```

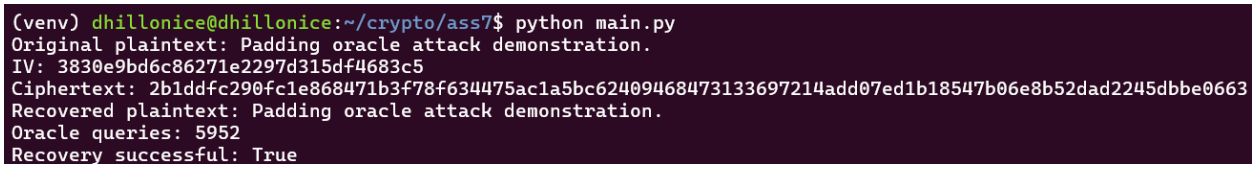


Figure 1: Execution of the Padding Oracle Attack and successful plaintext recovery.

7.2 Original Plaintext

The original plaintext provided to the encryption procedure was:

```
None
Padding oracle attack demonstration.
```

The plaintext was padded using PKCS#7 and encrypted using AES-CBC.

7.3 Initialization Vector

The Initialization Vector generated during execution was:

```
None
3830e9bd6c86271e2297d315df4683c5
```

The IV is 16 bytes long, corresponding to the AES block size.

In CBC mode, the IV is used when processing the first plaintext block.

7.4 Ciphertext

The ciphertext generated by the program was:

```
None
2b1ddfc290fc1e868471b3f78f634475ac1a5bc6240946847313369721 4add07ed1
b18547b06e8b52dad2245dbbe0663
```

The ciphertext is the encrypted representation of the padded plaintext.

The Padding Oracle Attack uses the ciphertext and IV to recover the plaintext without accessing the AES key during the attack.

7.5 Recovered Plaintext

The program successfully recovered the following plaintext:

```
None
Padding oracle attack demonstration.
```

The recovered plaintext matches the original plaintext.

This demonstrates that the attack successfully recovered the message using the padding oracle.

7.6 Number of Oracle Queries

The program reported the following result:

```
None
Oracle queries: 5952
```

A total of **5,952 oracle queries** were made during the plaintext-recovery process.

Each query represents an attempt to determine whether a modified ciphertext produces valid PKCS#7 padding.

The number of queries depends on the ciphertext length, the attack implementation, the search strategy, and the number of attempts required to recover individual bytes.

---

8. Analysis of the Attack

8.1 Why Does the Attack Work?

The attack works because CBC decryption uses the XOR operation between the decrypted ciphertext block and the preceding ciphertext block.

The decryption formula is:

$$\mathbf{Pi = DK(Ci) \oplus Ci-1}$$

If the attacker modifies **Ci−1**, the resulting plaintext block **Pi** changes.

The attacker can exploit this property by repeatedly modifying ciphertext bytes and submitting the modified ciphertext to the padding oracle.

When the oracle reports valid padding, the response provides information about the decrypted plaintext.

By systematically testing different values, the attacker can recover the plaintext bytes without knowing the AES key.

8.2 Why Is Plaintext Recovered from Right to Left?

PKCS#7 padding is located at the end of the final plaintext block.

For example, if five padding bytes are required, the last five bytes must all contain the value `05`.

The attack begins with the rightmost byte because it first attempts to create valid padding of length one.

After recovering the last byte, the attacker adjusts the preceding bytes to create padding of length two, then three, and so on.

This process continues until all bytes of the target block have been recovered.

8.3 Effect of Modifying the Previous Ciphertext Block

In CBC mode, each plaintext block depends on the decrypted current ciphertext block and the previous ciphertext block.

Therefore, modifying the previous ciphertext block affects the corresponding plaintext block after decryption.

The attacker uses this property to manipulate the plaintext bytes into candidate padding patterns.

The padding oracle indicates whether the resulting padding is valid, allowing the attacker to infer information about the original plaintext.

8.4 Interpretation of the Results

The execution produced the following results:

Parameter	Result
Original plaintext	Padding oracle attack demonstration.
Encryption mode	AES-CBC
Padding scheme	PKCS#7
IV length	16 bytes
Oracle queries	5952
Recovered plaintext	Padding oracle attack demonstration.
Recovery successful	True

The original and recovered plaintexts are identical.

The result demonstrates that the padding oracle provided sufficient information for the implemented attack to recover the plaintext without accessing the AES key during the attack.

The total number of oracle queries also illustrates that plaintext recovery requires repeated interactions with the oracle.

---

9. Security Recommendations

A Padding Oracle Attack can be prevented by designing encryption systems so that attackers cannot obtain useful information from padding-validation failures.

The following security measures can help prevent this vulnerability.

9.1 Use Authenticated Encryption

Authenticated encryption provides confidentiality and integrity.

Modern authenticated encryption algorithms, such as AES-GCM or ChaCha20-Poly1305, provide mechanisms for detecting unauthorized modifications to encrypted data.

Where possible, systems should use a well-implemented authenticated encryption mode rather than relying on unauthenticated AES-CBC encryption.

Authentication must be verified before decrypted plaintext is released or processed.

10. Verification

After the attack completed, the recovered plaintext was compared with the original plaintext.

The program produced the following verification result:

```
None
Recovery successful: True
```

The original plaintext was:

```
None
Padding oracle attack demonstration.
```

The recovered plaintext was:

```
None
Padding oracle attack demonstration.
```

Both strings match exactly.

Therefore, the program successfully recovered the plaintext using the padding oracle.

The result demonstrates that the implemented attack works for the provided test case.

---

11. Deliverables

The following deliverables were completed as part of the assignment:

1. **Python Program:** Implementation of AES-CBC encryption, the padding oracle, and the Padding Oracle Attack.
2. **Recovered Plaintext:** The original message was recovered successfully.

3. **Oracle Query Count:** The attack required 5,952 oracle queries.
4. **Attack Explanation:** The report explains AES-CBC, PKCS#7 padding, the padding oracle, and the plaintext-recovery process.
5. **Security Recommendations:** Preventive measures against padding oracle vulnerabilities are discussed.
6. **Execution Screenshot:** The terminal output demonstrates successful execution and plaintext recovery.

---

12. Conclusion

The Padding Oracle Attack was successfully implemented using Python to demonstrate how plaintext can be recovered from AES-CBC encrypted data without knowing the encryption key.

The experiment demonstrated the role of PKCS#7 padding and the importance of the padding oracle's responses in recovering plaintext bytes.

By modifying ciphertext blocks and repeatedly querying the padding oracle, the program recovered the original plaintext from right to left.

The recovered plaintext was:

```
None
Padding oracle attack demonstration.
```

The program reported a total of 5,952 oracle queries and confirmed successful recovery with the result:

```
None
Recovery successful: True
```

The experiment demonstrates that AES-CBC encryption can be vulnerable when an application exposes information about padding validity. Although the AES algorithm itself remains secure, an attacker can exploit weaknesses in the surrounding decryption process to recover plaintext.

The vulnerability can be mitigated by using authenticated encryption, verifying message authentication before decryption when using Encrypt-then-MAC, and preventing distinguishable error responses and timing leaks.

Thus, the assignment successfully demonstrates the working of a Padding Oracle Attack and highlights the importance of secure cryptographic implementation